06회차 문제 풀이 접근법



학습 목표 → 공식 근거 → 근본 원인 → 구현과 검증

01 / LEARNING GOALS

정답보다 먼저, 무엇을 입증할지 정합니다.

학습 목표	찾을 근거 → 판단 → 검증
Polling과 Interrupt의 선택	Main 루프와 STM Compare 경로 → 지연을 만드는 실행 주체 구분 → 흐름도와 동일 요청 시나리오.
짧은 ISR과 Main 분리	공식 STM ISR 및 Source 구성 → 재설정과 요청 기록만 ISR에 배치 → Sensor·Publish Context와 횟수 Test.
Race·Atomicity·Critical section	CPU Interrupt 제어와 읽기/초기화 순서 → 공유 상태를 하나의 Snapshot으로 이전 → Hook 재현·IRQ 이전 상태·경계 Test.

먼저 "요청을 보존한다"는 말을 구체화합니다. 이 과제는 실제 ISR 호출로 들어온 수락 요청을 최대 4개까지 셉니다. 네 개의 과거 센서값을 보관하지 않으며, 모든 Hardware 주기가 전달됐다고 보증하지도 않습니다.

01 요구 조건과 전제를 읽고, 실제 관찰값으로 실패를 재현합니다.

02 Datasheet·User Manual·공식 예제에서 Hardware 사실을 확인합니다.

03 그 사실 위에 제품 정책을 적용하고, 필요한 최소 공유 상태와 보호 범위를 정합니다.

04 각 근본 원인을 분리해서 검증하고 실패·성공 증거를 남깁니다.

공개 풀이 접근법입니다. 학습 목표와 공식 근거를 연결하고 원본의 11개 근본 원인을 모두 설명합니다. 내용을 참고해 직접 C 코드를 개선하고, 추가 Test와 실제 실행 결과를 자신의 PR에 남겨 주세요.

06회차 문제 풀이 접근법



학습 목표 → 공식 근거 → 근본 원인 → 구현과 검증

02 / DOCUMENT READING

Datasheet 하나로 실행 순서를 설명하지 않습니다.

근거	읽고 설명할 내용
TC38x Datasheet	TC387QP의 제품 계열과 STM 기능을 식별합니다. Datasheet는 이번 Host의 10ms·용량 4·최신값 병합 정책을 정해 주지 않습니다.
TC3xx User Manual Part 2 Chapter 27 STM	Counter가 free-running이며 Compare는 특정 Counter 값과 비교하는 구조임을 확인합니다. Compare 갱신을 Counter reset이라고 설명하면 다음 시간축을 잘못 이해하게 됩니다.
STM_Interrupt.c 공식 KIT_TC387_TFT 예제	Priority 40, CPU0 TOS, IFX_INTERRUPT 선언, IfxStm_increaseCompare 호출을 연결합니다. 예제의 숫자는 이 과제의 필수 Priority가 아닙니다.
IfxCpu.h / iLLD	disableInterrupts가 이전 enable 상태를 돌려주고 restoreInterrupts가 그 상태를 조건으로 복구함을 확인합니다. Critical section 안에서 임의 enable하지 않는 계약이 필요합니다.

STM Compare Flag와 Service Request Node의 pending 상태는 서로 다른 개념입니다. UM의 CMP0IR 설명은 이 Flag가 Interrupt 생성을 gate하지 않는다고 명시합니다. “Flag를 clear하지 않으면 무조건 폭주한다”는 설명은 여기서 성립하지 않습니다.

> 공식 Training PDF의 Counter reset이라는 요약 표현보다 UM의 Register 정의와 실제 increaseCompare 코드를 우선합니다. 절·함수·조건을 함께 적어 근거 간 표현 차이도 설명할 수 있어야 합니다.

공식 예제 고정 Commit

06회차 공식 자료와 문서 위치: REFERENCES.md

06회차 문제 풀이 접근법



학습 목표 → 공식 근거 → 근본 원인 → 구현과 검증

03 / EXECUTION MODEL

요청을 받는 것과 센서를 읽는 것은 다른 일입니다.

흐름	동작과 한계
Polling	Main이 시간/상태를 확인한 뒤 수집합니다. 구조가 단순하지만 Main의 느린 작업이 확인 시점을 늦춥니다. 반복 확인의 비용과 허용 지연을 함께 봅니다.
Interrupt	Hardware 요청으로 ISR이 실행되어 짧게 상태를 기록합니다. Main이 바빠도 IRQ가 허용되면 기록할 수 있지만, 차단이나 우선순위·ISR 지연을 제거하지는 못합니다.
이번 분리	Timer 서비스와 요청 기록은 ISR. Snapshot 이전만 Critical section. 센서값 읽기와 Publish는 Main·IRQ enabled 구간.

01 STM Compare → 요청 전달 → ISR에서 다음 Compare 서비스 → Mailbox 기록

02 Main → IRQ 상태 저장·차단 → Snapshot·초기화 → IRQ 상태 복구

03 유효 Batch이면 Sensor 1회 → Report 구성 → Publish 1회 → 다음 Main 반복

공식 예제의 previous Compare + period 방식은 기준 위상을 유지합니다. 그러나 이미 지나간 Compare 값을 늦게 설정하면 Hardware가 빠진 주기를 즉시 모두 보상한다고 할 수 없습니다. 기본 과제는 다음 Compare 설정이 10ms 이내에 완료된다고 가정합니다.

Main이 17ms 동안 IRQ를 허용하고 일하는 경우와 CPU가 17ms 동안 Interrupt를 차단하는 경우는 다릅니다. 전자는 Mailbox가 쌓일 수 있고 후자는 Interrupt 전달·다음 Compare 설정 자체가 늦어질 수 있습니다.

06회차 문제 풀이 접근법



학습 목표 → 공식 근거 → 근본 원인 → 구현과 검증

04 / ATOMIC SNAPSHOT

volatile은 읽기와 초기화를 묶어 주지 않습니다.

순서	보호되지 않은 읽기·초기화
1	공유 상태가 pending=1, last_tick=10인 상태에서 Main이 값을 읽습니다.
2	Main이 지우기 전에 ISR이 실행되어 새 요청 tick=20을 기록합니다.
3	Main이 공유 상태를 0으로 지웁니다. 새 요청은 반환한 Snapshot에도, 다음 Batch에도 남지 않습니다.

각 Load나 Store가 개별적으로 실행되더라도 여러 필드 읽기와 초기화 전체가 하나의 Transaction이 되는 것은 아닙니다. volatile은 최적화 관련 접근 특성을 표현하지만 상호 배제나 복합 연산의 Atomicity를 제공하지 않습니다.

```
saved = disable_interrupts();
local = snapshot_of_shared_fields;
platform_interleave();       // provided observation hook
if (local.pending > 0) { clear_shared_fields(); }
restore_interrupts(saved);
// A new ISR may run here. Never clear the mailbox here.
if (local.pending == 0) { return false; }
*out = local;
return true;
```

이전 IRQ가 enabled였다면 복구 시 보류된 ISR이 실행되어 tick=20이 새로운 Mailbox에 남습니다. 이전 IRQ가 disabled였다면 그대로 disabled여야 합니다. 함수가 자신의 호출자보다 넓은 권한으로 Interrupt를 켜면 바깥 Critical section의 보호가 깨집니다.

위 순서는 의사 코드입니다. 제공된 starter/include/mission.h와 platform.h의 API 계약에 맞춰 직접 구현합니다. 단일 CPU의 maskable ISR이라는 범위에서만 이 보호 전략을 적용합니다.

06회차 문제 풀이 접근법



학습 목표 → 공식 근거 → 근본 원인 → 구현과 검증

05 / CAPACITY & MEANING

수락·병합·거부를 서로 다른 수로 설명합니다.

ISR 입력	pending	last_tick	dropped
10	1	10	0
20	2	20	0
30	3	30	0
40	4	40	0
50	4	40	1
60	4	40	2

Main이 처리하면 requests=4, request_tick=40, coalesced=3, dropped=2입니다. Sensor는 현재 값을 한 번 읽습니다. requests 4개 중 한 번의 현재 수집으로 합친 나머지 3개가 coalesced이며, 마지막 두 개는 용량 초과로 수락하지 않았으므로 dropped입니다.

용량이 찼는데 last_tick을 60으로 바꾸면 Report의 요청 수와 시각이 서로 다른 수락 집합을 가리킵니다. pending만 늘리고 상한을 무시해도 처리 계약이 깨집니다. dropped가 UINT32_MAX일 때 더하지 않는 이유는 Overflow로 손실 개수가 0으로 보이는 것을 막기 위해서입니다.

이 Mailbox는 sample Queue가 아닙니다. 10ms마다 실제 센서값을 빠짐없이 보존해야 한다면 별도의 Timestamp 포함 sample Buffer와 수집 경로·처리량·Overflow 정책을 설계해야 합니다. 그 요구는 이번 과제의 최신값 병합 정책과 다릅니다.

06회차 문제 풀이 접근법



학습 목표 → 공식 근거 → 근본 원인 → 구현과 검증

06 / LATENCY REASONING

짧게 차단하고, 느린 작업은 Interrupt를 허용합니다.

분석용 가정	결과
ISR에서 Sensor 2ms + Publish 15ms	작업만 17ms로 주기 10ms보다 길어집니다. 다음 Compare 서비스까지 늦어지면 요청 지연과 누락 가능성이 생깁니다.
Main에서 IRQ disabled로 같은 작업	해당 maskable ISR이 적어도 이 작업 구간 동안 진입하지 못합니다. Counter는 계속 흐를 수 있으므로 CPU 차단과 Timer 정지를 혼동하지 않습니다.
Snapshot만 B 동안 차단	이 Critical section이 추가하는 차단은 최대 B입니다. 총 응답 지연은 다른 차단·우선순위·진입 비용 등을 포함하므로 B로 단정하지 않습니다.
Main에서 IRQ enabled로 같은 작업	ISR이 실행되어 다음 요청을 쌓을 수 있습니다. 현재 Report는 local Snapshot으로 유지하고 새 요청은 다음 Batch에서 처리합니다.

17ms는 Sensor와 Publish만의 비교 가정입니다. 원본 ISR의 별도 Delay 10ms까지 더하면 27ms에 기타 처리 시간이 추가됩니다. Host Delay는 Sleep하지 않고 호출만 기록합니다. B는 미측정 시간이며 Host 실행 시간으로 TC387QP의 B나 ISR Worst-case 실행 시간을 대신할 수 없습니다.

Board에서 추가로 확인할 증거

Clock·Compare 주기 설정, Compiler 최적화와 생성 명령, 최악 IRQ 차단 시간, ISR 진입부터 다음 Compare 갱신까지의 시간, 다른 Interrupt의 간섭을 확인합니다. 측정은 GPIO Toggle·Trace 등의 방법과 측정 위치를 기록해야 합니다.

> Critical section이 짧아졌다는 판단과 제품의 모든 Deadline을 만족한다는 판단은 별개입니다. Test로 입증한 범위와 아직 측정해야 할 범위를 각각 적습니다.

06회차 문제 풀이 접근법



학습 목표 → 공식 근거 → 근본 원인 → 구현과 검증

07 / TEST STRATEGY

오류 하나를 가리는 다른 오류를 분리합니다.

원본의 event_post_isr가 pending을 늘리지 않으면 Overflow Test가 Snapshot의 오류까지 도달하지 못할 수 있습니다. 따라서 Mailbox의 Snapshot Test는 유효한 초기 상태를 직접 준비하고, ISR Test와 Collector Test는 실행 Context·횟수·출력 보존을 각각 관찰합니다.

축	독립적으로 확인할 값
입력 계약	NULL과 Empty에서 false, 기존 out 보존, Mailbox 미소모, I/O 0회.
수량·경계	0·1·4·5개 요청, dropped 포화, 거부 시 last_tick 보존.
동시성 순서	읽기/초기화 사이 Hook, 복구 뒤 새 요청, 이미 disabled인 호출자, Sensor·Publish 중 새 요청.
Context	Timer 서비스 우선, ISR 안의 느린 I/O 없음, Main의 I/O는 IRQ enabled.
의미	requests와 coalesced 차이, 마지막 수락 시각과 측정 시각 차이, 현재 Batch와 다음 Batch 분리.

추가 Test는 예를 들어 tick=0인 유효 요청이나 꽉 찬 Mailbox를 비운 뒤 새 Batch의 dropped가 0에서 시작하는 상황처럼 별도의 가정을 흔들어야 합니다. 실행하지 않은 예상값을 실제 Log라고 쓰지 않습니다.

2026-10-17 macOS / Apple clang 21.0.0 / GNU Make 3.81 기준: 원본은 422 checks 중 106 failures, 내부 검증용 참고 구현은 추가 Test 포함 434 checks 모두 통과했습니다. 아래 원인별 설명의 변이 검증도 같은 환경의 결과입니다. Host 검증은 Board 실측이나 모든 동시 실행의 증명이 아닙니다.

06회차 문제 풀이 접근법



학습 목표 → 공식 근거 → 근본 원인 → 구현과 검증

08 / ROOT CAUSES

근본 원인 1-3: 실패에서 수정 근거까지

1. 요청 수를 1로 덮어쓴다

요구사항은 ISR이 실제로 실행될 때 수용 가능한 요청을 1씩 누적하는 것이다. 10·20·30에 요청을 보낸 뒤 Main이 실행되면 pending=3, last_tick=30이어야 한다. 초안 event_post_isr()는 매번 pending=1을 대입하여 pending=1만 남긴다. 이를 bool Flag처럼 취급한 것이 근본 원인이다.

pending 〈 EVENT_CAPACITY일 때 증가시킨다. 이 Counter는 과거 Sensor Sample 수가 아니라 처리 대기 중인 ISR 요청 수이다. test_post_policy의 2~4번째 요청 검사, test_end_to_end_burst에서 누락을 검출한다. Reference Demo의 late_main은 requests=3, coalesced=2, Sensor 1회이다.

2. 수용 한계와 Drop을 구분하지 않는다

10..60의 요청 6개를 Main이 처리하지 않았으면 수용 4개, 버린 요청 2개, 마지막 수용 시각 40이 남아야 한다. 초안은 pending=1, dropped=0, last_tick=60이다. Full 상태도 새로운 수용처럼 덮어써서 용량과 손실 정보가 사라진다. pending만 4로 막고 시각은 갱신하는 수정도 틀리다. 수용하지 않은 50·60 시각이 들어가기 때문이다.

Full이면 pending과 last_tick을 보존하고 dropped만 포화 증가시킨다. UINT32_MAX 다음 증가가 0으로 Wrap 되면 심한 손실을 손실 없음으로 보고하므로 상한을 검사한다. test_post_policy가 요청 4/5/6 경계와 UINT32_MAX-1, UINT32_MAX를 검사한다. 포화 검사 제거 변이는 2개 검사에서 실패했다.

3. Empty 반환에서 호출자의 결과를 파괴한다

Empty인 event_take()는 false와 출력 보존을 약속한다. out에 {91,92,93}을 넣고 빈 Mailbox를 가져오면 초안은 {0,0,0}으로 덮고 false를 반환한다. 성공 여부를 결정하기 전에 caller-owned 객체를 출력 버퍼로 사용한 것이 원인이다.

세 필드를 지역 EventBatch snapshot으로 먼저 읽는다. 유효한 Batch가 있을 때만 out에 대입한다. test_take_null_empty_and_previous_irq, test_snapshot_interleave의 Empty+새 IRQ Case가 출력 보존을 검사한다. NULL은 초안도 이미 안전하게 거부한다. NULL 방어를 새로운 결함으로 세지 않는다.

06회차 문제 풀이 접근법



학습 목표 → 공식 근거 → 근본 원인 → 구현과 검증

09 / ROOT CAUSES

근본 원인 4-5: 실패에서 수정 근거까지

4. 이전 IRQ 상태를 무시한다

호출 전 IRQ가 금지되어 있었다면 event_take 뒤에도 금지 상태여야 한다. 초안은 Save/Disable의 반환값을 버리고 restore(true)로 무조건 Enable한다. 바깥에서 유지 중인 Critical section을 내부 Helper가 종료하는 문제가 된다.

Save의 반환값을 보관하고 정확히 복원한다. 이전 Disabled Case에서 Hook IRQ가 보류된 채 남고, 바깥에서 Enable할 때 실행되는지 확인한다. test_take_null_empty_and_previous_irq, test_snapshot_interleave가 두 초기 상태를 검증한다. 무조건 Enable 변이는 7개 검사에서 실패했다. 이 과제는 Critical section 내부에서 다른 코드가 임의 Enable하지 않는다는 전제이다.

5. IRQ 복원 뒤에 공유 값을 지운다

시작 Mailbox가 {pending=4,dropped=1,last_tick=40}이고 Snapshot 뒤에 50의 IRQ가 도착한다고 하자. 초안은 읽는 동안 IRQ를 금지했으나 공유 Clear 전에 Enable한다. 기다리던 ISR이 새 요청을 기록한 뒤 Main이 세 필드를 0으로 지운다. 반환 Batch에는 이전 요청만 있고 다음 요청도 사라진다. 읽기만 보호해서는 Read-and-Clear 전체의 원자성이 보장되지 않는다.

Snapshot과 세 필드 Clear를 하나의 Critical section에 둔다. Clear 후 이전 상태를 복원하면 새 ISR은 빈 Mailbox에 다음 Batch의 요청을 적는다. 지역 Snapshot은 바뀌지 않으므로 IRQ 밖에서 out에 복사할 수 있다. volatile은 개별 메모리 접근의 의미를 제공할 뿐 이 복합 작업을 Transaction으로 만들지 않는다.

test_snapshot_interleave는 이전 Batch 반환과 다음 {1,0,50}의 보존을 함께 확인한다. Restore-before-clear 변이는 9개 검사에서 실패했다. Hook을 삭제하거나 위치를 옮겨 통과시키는 것은 수정이 아니다.

06회차 문제 풀이 접근법



학습 목표 → 공식 근거 → 근본 원인 → 구현과 검증

10 / ROOT CAUSES

근본 원인 6-8: 실패에서 수정 근거까지

6. ISR 안에서 Sensor·Publish·Delay를 실행한다

Timer ISR은 Timer 서비스와 Mailbox 전달만 해야 한다. 초안은 매 ISR에서 Sensor 1회, Publish 1회, Delay 1회를 실행한다. 따라서 6개 요청 뒤 Main을 실행하기 전에 이미 Sensor와 Publish가 6회씩 발생한다. Main도 다시 읽으므로 역할이 겹친다.

긴 작업을 ISR로 옮겨도 처리 시간 자체가 사라지지 않는다. 단일 비중첩 ISR에서는 다른 IRQ 처리가 지연될 수 있다. Timer ISR은 요청만 전달하고 Main에서 현재 Sensor를 한 번 읽는다. test_short_isr는 Sensor/Publish/Delay 0회, test_end_to_end_burst는 Main 전 I/O 0회를 검사한다. 고정 Host의 Delay는 실제 Sleep을 하지 않고 호출을 기록하므로 초안 Test가 멈추지 않는다.

7. Timer Service 호출 순서가 늦다

계약은 platform_timer_service()를 정확히 한 번 먼저 호출한 후 현재 Timer Tick을 읽고 요청을 게시하는 것이다. 초안은 Timer Now, Sensor, Publish, Delay, 요청 게시가 끝난 뒤 Service를 호출한다. Trace는 정상 ISNX 대신 INRPDSX이다.

Service를 첫 작업으로 옮긴다. Host는 호출 순서만 검사하며 이 함수의 실제 Target 구현은 다음 Compare 준비 등을 포함하는 별도 작업이다. 특정 CMP Flag를 지우지 않으면 반드시 폭주한다고 단정하지 않는다. 실제 STM Compare 요청과 Status Bit의 의미는 공식 User Manual로 따로 확인해야 한다. test_short_isr가 순서와 호출 횟수를 검사한다.

8. Main의 긴 I/O까지 IRQ를 금지한다

Critical section이 필요한 공유 데이터는 Mailbox의 Snapshot/Clear이다. 초안 collector_step()는 이미 Batch를 지역 변수로 받은 뒤 다시 IRQ를 금지하여 Sensor와 Publish를 실행한다. 경쟁을 막아야 할 공유 데이터 접근이 끝났는데도 긴 작업을 금지 구간에 넣은 것이 원인이다.

event_take가 복원한 IRQ enabled 상태에서 Sensor/Publish를 실행한다. 이때 들어온 ISR은 다음 Step용 Mailbox에 요청을 남긴다. test_collector_contract는 io_irq_disabled=0을 검사하고 test_new_irq_during_main_io는 Sensor/Publish 중 새 요청이 보존되는지 확인한다. 짧은 ISR만으로 실제 10 ms Sampling이 보장되는 것은 아니다. Main의 서비스 시간과 최악 지연도 별도로 분석해야 한다.

06회차 문제 풀이 접근법



학습 목표 → 공식 근거 → 근본 원인 → 구현과 검증

11 / ROOT CAUSES

근본 원인 9-11: 실패에서 수정 근거까지

9. Coalesced 수가 한 개 많다

요청 3개를 하나의 현재 Sample로 처리하면 요청 1개에 대응하는 Sample을 읽고 나머지 2개를 합친다. 초안은 coalesced=3을 넣어 모두 합쳐 처리한 것처럼 계산한다. 유효 Batch의 pending은 1..4로 보장되므로 pending-1이 맞고 Underflow도 없다.

test_collector_contract가 Batch 크기 1..4를 각각 검사한다. 가장 작은 Batch 1개는 coalesced=0이어야 한다. 한 번 읽은 값을 과거 요청 수만큼 복제하거나 현재 Sensor를 여러 번 읽는 것은 과거 Sampling을 복원하지 못한다. Off-by-one 변이는 20개 검사에서 실패했다.

10. 요청 시각을 Main 처리 시각으로 바꾼다

마지막으로 수용한 요청 Tick이 30이고 Main이 500에서 실행되면 request_tick=30이어야 한다. 초안은 platform_timer_now()를 다시 호출하여 500을 기록한다. 요청이 생긴 시각과 실제 처리를 시작한 시각을 같은 의미로 취급한 것이 원인이다.

Report에는 Batch의 last_tick을 그대로 넣는다. 이 값은 마지막 수용 ISR Timestamp이며 실제 Sensor Sampling 시작 시각도 아니다. 별도의 Sampling 시각이 필요하면 다른 필드와 측정이 필요하지만 이 과제의 API 변경 대상은 아니다. test_collector_contract는 요청 Tick과 Main Tick을 일부러 다르게 주고, 추가 Test는 Tick 0도 유효함을 확인한다.

11. Drop 정보를 Report에서 버린다

Batch가 {4,2,40}이면 Report의 dropped는 2이다. 초안은 항상 0을 대입하여 이전 단계가 알아낸 손실을 최종 결과에서 숨긴다. Mailbox의 Overflow 처리만 고쳐도 Report까지 전달되지 않으면 관찰자는 손실을 알 수 없다.

report.dropped = batch.dropped로 전달한다. test_collector_contract의 Full Batch Case와 test_end_to_end_burst가 확인한다. 추가 Test는 Overflow Batch를 가져가는 도중 새 IRQ가 들어와도 이전 dropped=2가 새 Batch의 dropped=0과 섞이지 않음을 검증한다.

06회차 문제 풀이 접근법



학습 목표 → 공식 근거 → 근본 원인 → 구현과 검증

12 / QUESTIONS & COMPLETION

개선 이유를 직접 설명할 수 있어야 합니다.

uint32_t라면 읽기 하나는 Atomic하지 않나요?

그 여부를 가정해도 세 필드 읽기와 초기화 전체가 Atomic해지는 것은 아닙니다. 이번 오류는 여러 접근 사이에 ISR이 들어올 수 있다는 점입니다.

Interrupt를 쓰면 모든 10ms 요청을 보존하나요?

아닙니다. CPU 차단과 우선순위, ISR 길이, Compare 갱신 지연, Hardware pending 표현에 영향을 받습니다. 여기의 dropped는 실제 호출된 ISR 중 Mailbox가 거부한 수만 셉니다.

왜 requests만큼 Sensor를 여러 번 읽지 않나요?

나중에 여러 번 읽어도 이미 지나간 시각의 값이 복원되지 않습니다. 이번 요구는 최신값 한 번과 병합 개수 보고입니다. 과거 값이 필요하면 수집 시점부터 Buffer에 저장해야 합니다.

왜 모든 함수에서 disable / enable하면 안 되나요?

느린 작업까지 차단하면 지연이 커집니다. 또 이전 상태를 무시한 enable은 호출자가 이미 만든 Critical section을 깨뜨립니다. 보호할 공유 상태 이전에만 적용하고 저장한 상태를 복구합니다.

완료 후 확인: Polling과 Interrupt의 사용 조건을 설명했다. ISR과 Main의 역할을 나눴다. Race를 실제 순서로 재현했다. 이전 IRQ 상태와 새 요청을 보존했다. Datasheet·UM·예제 근거, 모든 원인, Test와 지연 분석을 연결했다.